ME 133b Monte Carlo Localization Report

Felipe Cruz*[a,2] and Rupali Batta†[b,1]

[a]Caltech
[b]Caltech

Abstract—We implement Monte Carlo localization (MCL) for a robot moving in a 2D environment with known, time-varying obstacles. The filter represents the belief over the robot's pose as a weighted particle set, propagates particles with a motion model driven by control inputs, updates weights via a Gaussian measurement model on simulated four-beam lidar, and resamples using low-variance resampling with roughening and random particle injection. We demonstrate reliable tracking under motion and sensing noise and recovery from a kidnapped-robot event. This report summarizes the problem, algorithm design choices, technical details (state representation, motion and measurement models, resampling), parameter roles and trade-offs, and lessons learned.

Keywords—*Monte Carlo localization, particle filter, robot localization, kidnapped robot, resampling*

1. OVERVIEW

The goal of this project is to estimate the pose of a robot in a known 2D map using only control inputs and noisy range (lidar) measurements—a classic problem in mobile robotics. The map is a bounded rectangle with polygonal obstacles that move over time (e.g., drifting), so the environment is dynamic. The robot does not have access to its true position; it receives a velocity command (e.g., from a human via WASD) and a simulated four-direction lidar scan. The challenge is to maintain a belief over the robot's (x, y) position that stays concentrated near the true pose despite motion noise, lidar noise, and moving obstacles. A secondary challenge is *recovery from kidnapping*: at a chosen time step the robot is teleported to a random free position to simulate a catastrophic localization failure; the filter must re-localize using only subsequent controls and measurements.

Monte Carlo localization (MCL) is well-suited to this setting because the belief can be multi-modal (e.g., after kidnapping), and the motion and measurement models are nonlinear and non-Gaussian. We represent the belief as a set of N weighted particles $p_i = (x_i, y_i, w_i)$ with $\sum_i w_i = 1$. Each particle is a hypothetical position; the weighted set approximates the posterior. The algorithm alternates: (1) *predict* by propagating each particle with the motion model, (2) *update* by assigning weights proportional to the measurement likelihood, and (3) *resample* to obtain an unweighted set that concentrates particles in high-probability regions. The state estimate is the mean of the particle positions (or the weighted mean before resampling). Below we explain why this approach works, the specific choices we made, and how parameters affect behavior.

2. APPROACH

2.1. Why a particle filter

The posterior over robot pose given controls and measurements is not Gaussian: the motion model has bounded support (collisions keep the robot in free space), and the measurement model involves raycasting and noise. A Kalman filter or EKF would approximate the posterior as Gaussian and can perform poorly with multi-modal beliefs or strong nonlinearities. A particle filter, instead, has a sampled representation of the posterior; as N grows it converges to the true posterior under mild conditions. For real-time use we keep N modest (e.g., 25–500) and rely on resampling to keep particles where the likelihood is high.

2.2. High-level pipeline

At each time step t:

1. **Motion update:** For each particle, sample a new position from $p(x_t \mid x_{t-1}, u_t)$ using the control u_t and the motion model (with collision checking).
2. **Measurement update:** Compute the weight $w_i \propto p(z_t \mid x_i)$ for each proposed particle using the current lidar measurement z_t and the measurement model.
3. **Resample:** Draw N new particles from the weighted set (low-variance resampler) with optional roughening and random injection.
4. **Estimate:** Set the state estimate to the mean of the resampled particle positions.

Obstacles are updated on a fixed timer (e.g., every 100 ms) so the map changes continuously; the filter uses the current obstacle configuration for raycasting and collision checks at each step.

3. TECHNICAL DETAILS

3.1. State and environment

State: Each particle is a 2D point (x_i, y_i) with weight w_i. The full state is the set $\{(x_i, y_i, w_i)\}_{i=1}^N$ with $\sum_i w_i = 1$. The map is the rectangle $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$ (e.g., $[0, 10] \times [0, 12]$) minus the union of four polygonal obstacles. Obstacles move: obstacle k is translated by $(\rho_k \cos(\omega_k t + \phi_k), \rho_k \sin(\omega_k t + \phi_k))$ so the free space $\mathcal{X}_{\text{free}}(t)$ is time-dependent. A point (x, y) is in free space iff it lies in the rectangle and is disjoint from all current obstacle polygons (implemented with Shapely).

3.2. Motion model

The robot receives a control $u_t = (v_x, v_y)$ (velocity). We model:

$$x_t = x_{t-1} + v_x \Delta t + n_x, \quad y_t = y_{t-1} + v_y \Delta t + n_y, \tag{1}$$

with $n_x, n_y \sim \mathcal{N}(0, \sigma_{\text{bot}}^2)$ i.i.d. (e.g., $\sigma_{\text{bot}} = 0.01$). If the resulting (x_t, y_t) is not in free space, we keep (x_{t-1}, y_{t-1}). Thus the transition is a truncated Gaussian. For each particle we sample one successor; the proposal is drawn from this motion model (sample-motion-model approach in MCL).

3.3. Lidar and measurement model

Lidar simulation: From a point (x, y), we cast rays in four directions $\{(1, 0), (0, 1), (-1, 0), (0, -1)\}$. For each direction (d_x, d_y), we march along the ray in steps of size δ until we hit the boundary or an obstacle; the reported range is the distance to the first hit, capped at $r_{\max}$ (e.g., 7.5). The "true" robot receives this range plus Gaussian noise $\mathcal{N}(0, \sigma_{\text{lidar}}^2)$ per beam (e.g., $\sigma_{\text{lidar}} = 1$), then clipped to $[0, r_{\max}]$.

Measurement likelihood: For a particle at (x_i, y_i) we compute the *predicted* four-beam scan $\hat{z}$ (no noise). We model the measurement likelihood as a product of independent Gaussians (one per beam):

$$p(z \mid x_i) = \prod_{j=1}^{4} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(z_j - \hat{z}_j)^2}{2\sigma^2}\right), \tag{2}$$

with $\sigma = \sigma_{\text{lidar}}$. Particles in free space that predict a scan similar to z receive high weight; particles inside obstacles get weight 0 (we return 0 if the particle is not in free space).

3.4. Resampling

We use *low-variance resampling* (systematic resampling): draw one random offset $r \sim \text{Uniform}(0, 1/N)$, then select indices $i_1, \ldots, i_N$ so that the cumulative sum of normalized weights is traversed with step $1/N$ starting at r. This reduces variance compared to multinomial

*fcruzfal@caltech.edu
†rupali@caltech.edu

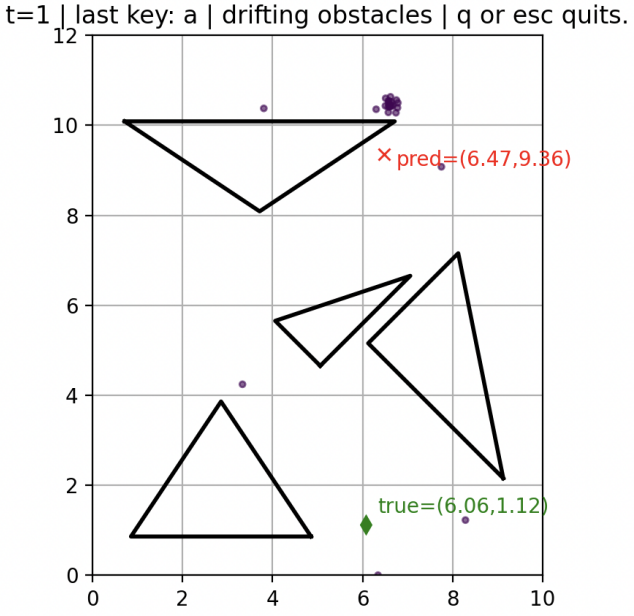


Figure 1. Example of a the MCL visualization.

resampling and keeps diversity. After selecting a source particle we add Gaussian jitter (roughening) with standard deviation ϵ (e.g., 0.1) to the position; if the jittered point is in collision we keep the original position. With probability p_{rand} (e.g., 0.1) we instead inject a particle uniformly at random in free space. Roughening prevents particle collapse; random injection helps recovery from kidnapping. If the total weight is zero (all particles in collision or inconsistent with the measurement), we reinitialize N particles uniformly in free space.

Effective sample size (ESS): We compute $\text{ESS} = 1/\sum_i w_i^2$ (after normalizing weights). ESS is at most N; if ESS is small, most of the mass is on few particles and we are at risk of losing diversity. Logging ESS and max weight helps tune N, σ_{lidar}, and p_{rand}.

3.5. Kidnapped-robot experiment

At a fixed step $t = T_{\text{teleport}}$ (e.g., 60), the true robot position is replaced by a random point in free space; the particle set is unchanged. (Reduce `TELEPORT_STEP` in the notebook to see the effect sooner.) So at that instant the estimate is wrong. Without random injection, the filter would likely never recover because all particles would remain near the old pose. With a fraction p_{rand} of particles drawn uniformly in free space at each resample, some particles will eventually land near the new pose and get high weight, and the belief recenters. The number of steps to re-localize depends on p_{rand}, N, and the map.

4. CONTRIBUTIONS AND LESSONS LEARNED

4.1. Parameter roles and trade-offs

- σ_{lidar}**:** Larger values make the measurement likelihood flatter, so particles farther from the "correct" scan still get non-negligible weight; this can help with noise but blurs the belief. Smaller σ sharpens the likelihood and can lead to premature particle collapse if the model is wrong or noise is understated.
- σ_{bot} **(motion noise):** Slightly inflating motion uncertainty spreads particles after the predict step and can improve robustness; too much spread makes the belief too diffuse.
- **Roughening** ϵ**:** Prevents many particles from sitting at the same point after resampling. Too small and we lose diversity; too large and we inject unnecessary uncertainty.
- **Random fraction** p_{rand}**:** Critical for kidnapping recovery. A typical value is 5–10%. Higher values improve recovery but can make the estimate noisier when the filter is already well localized.
- **Particle count** N**:** More particles give a better approximation to the posterior and smoother estimates but increase compute (motion + N raycasts per step). With 25 particles we can track and recover; 100–500 is more robust for harder maps.

4.2. Why low-variance resampling

Multinomial resampling draws N independent indices from the categorical distribution over particles; the number of copies of each particle has high variance. Low-variance (systematic) resampling ensures that if a particle has weight w_i, it is copied approximately Nw_i times with minimal variance. This preserves diversity and reduces the chance that a single high-weight particle fills the entire set in one step.

4.3. Limitations and extensions

The current setup uses four axial lidar beams; adding more beams or angles would improve discriminability of poses. The map is known and obstacles are polygons; extending to occupancy grids or unknown obstacles would require a different representation. We do not estimate orientation; adding θ would triple the state dimension and typically require many more particles or a hybrid representation. For very large maps, global localization can be accelerated by initializing particles from a coarse grid or using a separate place-recognition module to propose regions.

5. EXPERIMENTAL RESULTS

We ran batch experiments with a fixed control sequence (120 steps, teleport at step 60) and report mean localization error (Euclidean distance from estimate to true pose), time per MCL step, and effective sample size (ESS) where relevant. Each setting was run over 5 trials unless noted. Figures are saved in `mcl_experiment_results/figures/`; tables below summarize the data.

5.1. Particle count: error and compute trade-off

Table 1 shows mean error before and after the kidnapping event, and mean time per step, for $N = 10, 25, 50, 100, 200, 500$. Error improves as N increases from 10 to 50; beyond 100, error is similar while time grows roughly linearly with N. Fig. 2 plots error vs step for $N = 10, 25, 100$, with the teleport at step 60. Fig. 3 compares mean error before vs after kidnap for all N. Fig. 4 illustrates the compute trade-off: time per step scales with N.

Table 1. Error and time vs number of particles (5 runs each).

N	Mean error (before)	Mean error (after kidnap)	Time/step (ms)
10	4.21 ± 1.40	2.96 ± 3.43	13.4 ± 0.9
25	4.16 ± 1.56	2.46 ± 1.04	38.1 ± 10.3
50	0.95 ± 0.27	1.10 ± 0.50	71.7 ± 8.5
100	1.17 ± 0.36	2.51 ± 0.85	129.0 ± 8.6
200	1.03 ± 0.41	0.90 ± 0.31	307.1 ± 24.3
500	0.79 ± 0.07	1.04 ± 0.38	858.8 ± 115.4

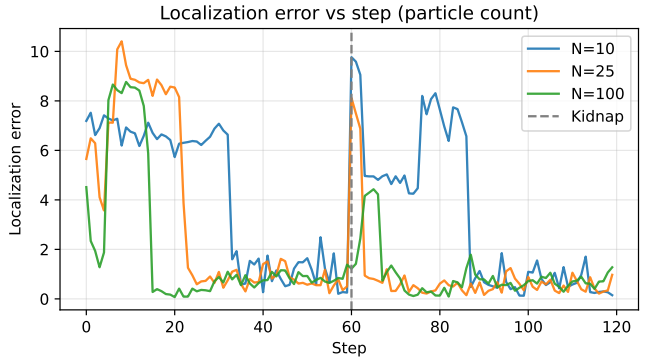


Figure 2. Localization error vs step for $N = 10, 25, 100$. Vertical line: kidnap at step 60.

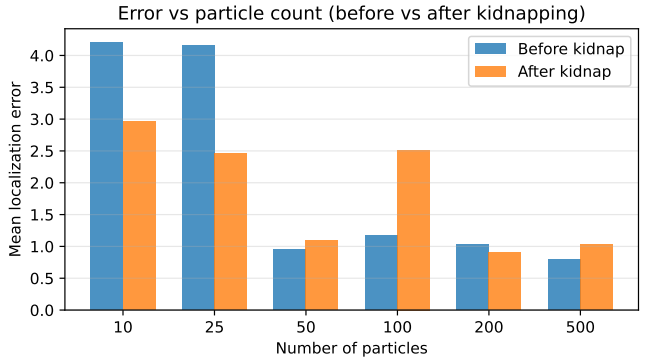


Figure 3. Mean error before vs after kidnapping for each particle count.

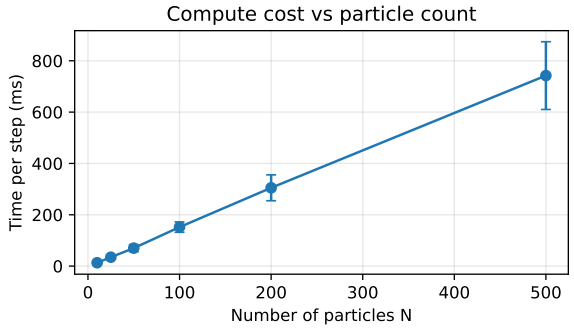


Figure 4. Time per MCL step vs N. Compute cost grows with particle count.

Interpretation: With only 10 or 25 particles, the belief is too coarse: mean error before kidnap stays around 4.2 and is highly variable (large std). Once N reaches 50, the filter approximates the posterior much better and mean error drops to about 1.0. Further increasing N to 200 or 500 yields modest additional gains in accuracy (mean error $\approx$ 0.8–1.0) but time per step grows from ~70 ms to ~860 ms. So there is a clear trade-off: too few particles give poor localization; too many give diminishing returns at high cost. A practical choice is $N \in [50, 200]$ for this map and sensor setup. The higher variance in "after kidnap" error for $N = 10, 25$ also reflects that with few particles, recovery is inconsistent—sometimes a few particles land near the new pose by chance, sometimes they do not.

5.2. Lidar noise sensitivity

Table 2 and Fig. 5 show that mean error (before kidnap) increases as the measurement noise σ used in the likelihood increases: $\sigma = 0.5$ gives lowest mean error; $\sigma = 2$–3 gives higher error. Matching σ to the true sensor noise (or slightly overestimating it) is important for good localization.

Table 2. Sensitivity to lidar noise σ (mean error before kidnap).

σ (lidar noise)	Mean error (before kidnap)
0.5	1.05 ± 0.62
1.0	2.49 ± 1.62
2.0	3.47 ± 1.29
3.0	3.28 ± 0.51

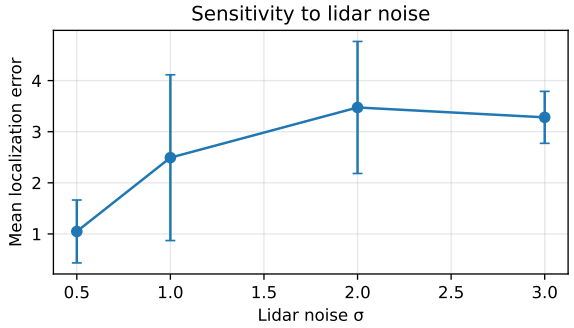


Figure 5. Mean localization error vs lidar noise σ.

Interpretation: The measurement likelihood uses σ to model sensor noise. When $\sigma = 0.5$ (tighter than or close to the true noise in our simulation), the filter trusts the measurement more and concentrates particles where the predicted scan matches the observation, giving the lowest mean error (1.05). As σ increases to 1, 2, or 3, the likelihood becomes flatter: particles that predict scans farther from the observation still get non-negligible weight. The belief is therefore more diffuse and the mean estimate is less accurate (error rises to 2.5–3.5). So the results support that the algorithm is sensitive to this parameter and that setting σ to match (or slightly underestimate) the actual sensor noise yields the best localization. Overestimating σ is a conservative choice that can help in very noisy settings but here it degrades accuracy.

5.3. Roughening and ESS

Table 3 reports mean error and mean ESS for roughening $\epsilon = 0, 0.05, 0.1, 0.2$. Higher roughening slightly reduces mean error (1.96–2.10) and lowers mean ESS (13–19), indicating more spread and diversity. Fig. 6 shows mean ESS vs ϵ.

Table 3. Roughening vs mean error and mean ESS.

Roughening ϵ	Mean error	Mean ESS
0.0	2.59	19.0
0.05	2.65	16.9
0.1	2.10	15.3
0.2	1.96	13.0

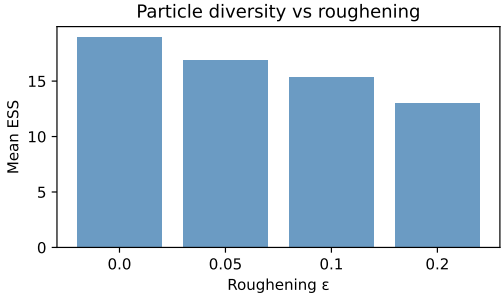


Figure 6. Mean effective sample size vs roughening.

Interpretation: With no roughening ($\epsilon = 0$), resampling can place many particles at the same or nearly the same location, so ESS is high (19) but the cloud has little spatial diversity—if the true pose shifts, the filter may be slow to follow. As ϵ increases, we add jitter to resampled positions, which spreads particles out and reduces ESS (down to 13 for $\epsilon = 0.2$), but mean error actually improves (2.59 → 1.96). So a moderate amount of roughening (0.1–0.2) both maintains diversity and improves accuracy by avoiding particle collapse. The results indicate that some roughening is beneficial and that ESS alone is not a direct stand-in for localization quality: slightly lower ESS with more spread can be better than high ESS with all mass in one spot.

5.4. Resampling: low-variance vs multinomial

Table 4 compares low-variance and multinomial resampling. Low-variance gives slightly lower mean error before kidnap (3.35 vs 3.53) and notably lower mean error after kidnap (2.59 vs 3.99). Fig. 7 shows error vs step for one run of each; low-variance recovers more consistently after the teleport.

Table 4. Low-variance vs multinomial resampling.

Resampler	Mean error (before)	Mean error (after kidnap)
Low-variance	3.35	2.59
Multinomial	3.53	3.99

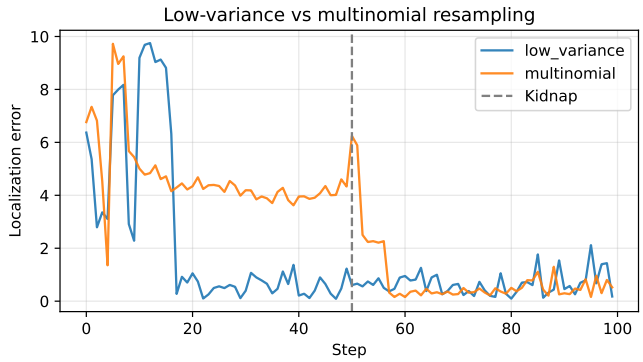


Figure 7. Localization error vs step: low-variance vs multinomial resampling.

Interpretation: Low-variance (systematic) resampling produces a more representative set of copies: high-weight particles get roughly Nw_i copies with low variance, so diversity is preserved. Multinomial resampling, by drawing N independent indices, has higher variance in the number of copies per particle; a single particle can be over-sampled and dominate the set. The table and figure show that low-variance gives better mean error both before and after the kidnap (3.35 vs 3.53 before; 2.59 vs 3.99 after). The larger gap after kidnap indicates that low-variance resampling helps the filter recover more reliably, because the particle set does not collapse onto one or a few locations and retains enough spread to eventually place particles near the new true pose. So for this MCL setup, low-variance resampling is the preferred choice.

5.5. Kidnapping recovery and random particle injection

Table 5 shows that without random particle injection (frac = 0), mean error after kidnap is 5.10; with frac = 0.1 it drops to 1.67. Fig. 8 plots error vs step after the teleport: with injection the filter recovers; without it the estimate remains poor.

Table 5. Kidnapping recovery: mean error after teleport.

Random frac p_{rand}	Mean error (after kidnap)
0.0	5.10
0.1	1.67

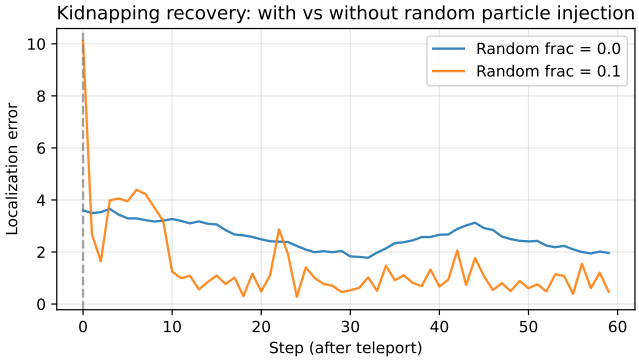


Figure 8. Error after teleport: with (frac = 0.1) vs without (frac = 0) random particle injection.

Interpretation: When the robot is teleported, all particles still sit near the old pose; without any mechanism to propose hypotheses elsewhere, the filter has no way to recover. With random fraction $p_{\text{rand}} = 0$, mean error after kidnap stays high (5.10) and the curve in Fig. 8 remains elevated. With $p_{\text{rand}} = 0.1$, at each resample we inject 10% of particles uniformly in free space, so some will eventually fall near the new true position and receive high measurement weight; the belief recenters and mean error drops to 1.67. The results show that a small fraction of random particles is necessary and sufficient for re-localization after a kidnapping event, and that without it the filter is effectively stuck with a wrong belief. This justifies the design choice of including random injection in the resampling step.

6. CONCLUSIONS

We implemented a particle filter for 2D robot localization in a known, dynamic environment with simulated lidar. The core design—sample-motion model, Gaussian measurement likelihood on predicted scans, low-variance resampling with roughening and random particle injection—yields stable tracking and recovery from the kidnapped-robot scenario. Key takeaways are: (1) the measurement model must match the actual noise (or be slightly conservative) to avoid collapse; (2) a small fraction of random particles is essential for global re-localization; (3) ESS and max weight are useful diagnostics for tuning. The project goes beyond a basic MCL by including moving obstacles, collision handling in the motion model, and an explicit kidnapping test with recovery, and by exposing parameters (ROUGHENING, RANDOM_PARTICLE_FRAC, LIDAR_NOISE) so their effect can be studied. For anyone replicating this, we recommend starting with the default parameters, then reducing T_{teleport} to observe kidnapping sooner and varying p_{rand} and N to see recovery speed and estimate quality.